

Advanced Software Design / Reengineering Project

Legacy University Course Registration System: Assessment, Optimization,
Reengineering, Security, and Validation

Prepared by: Ramya Mercy Rajan and Dheeraj Erpa

Course: Advanced Software Design

Instructor: Dr. Mohamed Nazeh Alimam

Executive Summary

This report presents a consultancy assessment of the university's existing Course Registration System, which is currently built as a legacy monolithic application. The system is used by students, lecturers, administrative staff, and the finance department. However, it experiences several issues, including slow performance during peak enrollment periods, database inconsistencies, security vulnerabilities, and usability problems.

The report follows the required modelling lifecycle, including use case modelling, process modelling, alignment, build, accuracy, benchmarking, testing, release, performance measurement, and consistency/reliability.

It is divided into five main tasks:

- (1) analysing the current system and identifying its problems,
- (2) proposing optimization techniques,
- (3) reengineering the system into a microservice architecture,
- (4) improving security and reliability, and
- (5) testing, validating, and evaluating the proposed solution.

Based on the analysis, the main recommendation is to replace the monolithic system with a microservice architecture consisting of five independent services: Authentication, Enrollment, Payment, Notification, and Academic Record. Each service will have its own database and will communicate through an API Gateway and an event-based messaging system. Security will be strengthened by implementing Role-Based Access Control (RBAC), Multi-Factor Authentication (MFA), data encryption during transmission and storage, and continuous system monitoring. In addition, database optimization, caching, and a DevSecOps deployment pipeline are proposed to improve system performance, reduce single points of failure, and ensure the system can handle high enrollment traffic efficiently.

Introduction

Background

The university's Course Registration System was developed more than ten years ago as a single monolithic application. Over time, new features and functions have been added to meet changing requirements. Today, the system plays an important role in the university, especially during the course enrollment period each semester. However, because it was not originally designed to support a large number of users at the same time or meet modern security and availability requirements, it now faces several challenges. To address these issues, the university has assigned this consultancy project to evaluate the current system, identify its weaknesses, recommend optimization techniques, and propose a redesigned architecture that better meets current and future needs.

Objectives

This consultancy project has five main objectives based on the tasks provided in the assignment. The first objective is to analyse the current system and identify its technical and operational problems using evidence from the assessment. The second objective is to recommend short-term optimization techniques that can improve the existing system without completely replacing it. The third objective is to design a new microservice-based architecture as the long-term solution for the system. The fourth objective is to include appropriate security and reliability measures in the proposed design. Finally, the fifth objective is to explain how the performance and effectiveness of the redesigned system can be tested and compared with the existing legacy system.

Scope

The scope of this project covers the entire course registration process, including authentication, course catalog management, course enrollment, tuition payment, notifications, and academic records. Systems that are not part of this domain, such as HR, alumni relations, and campus facilities, are not included, except where they need to be considered as external integration points.

This engagement is focused on analysis and design rather than implementation. It provides a detailed assessment of the current system, a proposed target architecture, and a validation plan that could later be used by a development team to build the actual system.

Assumptions

Since the brief describes a general scenario rather than a fully detailed real-world system, several assumptions are made to support the analysis and design. These assumptions also help ensure that the design decisions can be clearly linked back to the given context.

The existing legacy system is assumed to be a web-based application that uses a single relational database such as MySQL, PostgreSQL, or SQL Server.

The university is assumed to have a student population of around 20,000 to 40,000 students, similar to a large public university, with most system load occurring during short peak enrollment periods at the beginning of each term.

It is assumed that the finance department needs to verify and reconcile student payments with course enrollments, meaning the Enrollment and Payment systems are closely dependent on each other.

The university is also assumed to follow standard data protection and academic record regulations similar to FERPA, even if a specific legal framework is not mentioned in the brief.

No direct access to the real system, logs, or performance data was available. Therefore, the identified issues are based on the scenario description and common problems seen in similar systems rather than real measured data.

Report Organization

This report is structured into five main tasks: Task 1 (system assessment), Task 2 (optimization), Task 3 (reengineering), Task 4 (security and reliability), and Task 5 (testing, validation, and evaluation). This is followed by the final recommendations and a references list.

Modelling Lifecycle Applied in This Report

The assignment defines a ten-stage modelling lifecycle, which includes use case modelling, process modelling, alignment (correctness), build, accuracy, benchmarking and validation, testing, release, performance measurement, and consistency/reliability. These stages are grouped across four phases: Alignment, Realization, Verification, and Operation, as shown in the brief's diagram.

The table below explains how each stage is applied within this report.

Lifecycle Stage	Phase	Applied In This Report
1. Use Case Modelling	Alignment	Task 1.5 – Use Case Diagram; Task 1.1 stakeholder overview (students, lecturers, administrators, finance staff)
2. Process Modelling	Alignment	Task 1.5 – Activity Diagram of the registration and payment workflow
3. Correctness (Alignment)	Alignment	Comparison of Use Case and Activity diagrams; Task 3.1 bounded-context mapping based on original use cases
4. Build / Develop	Realization	Task 2 optimized schema, SQL, and pseudocode; Task 3 microservice architecture, API specification, and DevSecOps pipeline
5. Accuracy	Realization	Task 5.3 Validation report checking whether the design meets the defined objectives
6. Benchmarking & Validation	Verification	Task 5.3 Validation report and Task 5.4 performance comparison metrics
7. Testing	Verification	Task 5.2 test checklist (functional, integration, regression, performance, security, and reliability testing)
8. Release & Deployment	Operation	Task 3.5 DevSecOps pipeline proposal (CI/CD, containerization, and progressive delivery)
9. Performance Measurement	Operation	Task 5.4 performance comparison metrics (based on projected KPIs)
10. Consistency & Reliability	Operation	Task 4.4 fault tolerance and recovery; Task 4.5 incident response checklist

Table of Contents

Introduction	1
Modelling Lifecycle Applied in This Report	2
Task 1 – System Assessment and Problem Analysis	3
Task 2 – Optimization Design and Improvement	4
Task 3 – Software Reengineering and Architecture Redesign	5
Task 4 – Security and Reliability Engineering	6
Task 5 – Testing, Validation, and Evaluation	7
Final Recommendations and Future Improvements	8
References	9

Task 1 – System Assessment and Problem Analysis

1.1 Current System Overview

The existing Course Registration System is a monolithic web application built as a single-tier system. It is deployed as one application and uses a single relational database. All core functions, including authentication, course catalog management, enrollment, payment processing, grading, and notifications, are implemented as modules within the same application.

Since everything runs within a single system and shares the same database and connection pool, all operations are tightly coupled. This means that changes or issues in one part of the system can easily affect other parts as well.

1.2 Deliverable 1: Problem Analysis Report

The consultancy team identified several technical and operational issues in the current system. These issues are grouped into different categories for clarity.

Performance Issues

- The system runs on a single application server, meaning all traffic is handled in one place without horizontal scaling or load balancing between modules.
- During enrollment periods, when a large number of students register within a short time (around 48 hours), the system experiences CPU overload and connection pool exhaustion.
- Communication between modules is synchronous and blocking (for example, enrollment processes wait for payment confirmation within the same application), which increases response time.
- There is no caching mechanism for frequently accessed but rarely changed data such as course catalogs and program requirements.

Database Issues

- Important foreign key fields such as `student_id` and `course_id` are not properly indexed, leading to full table scans during peak usage.
- Some student and course data is duplicated across multiple tables, which results in update inconsistencies.
- Long-running transactions during seat allocation create lock contention and can lead to deadlocks.
- The system does not use read replicas, so reporting queries and live enrollment transactions compete for the same database resources.

Security Issues

- Passwords are assumed to be stored with weak hashing or without proper salting, and multi-factor authentication is not implemented.
- Role-based access control is not applied consistently, with some permissions enforced only at the UI level instead of the server side.
- Sensitive data such as personal identifiers and payment information is not consistently encrypted at rest, and internal communication does not always use TLS.
- Audit logging is insufficient, making it difficult to reliably track actions such as grade changes or payment adjustments.

Scalability Issues

- Because the system is monolithic, any small change requires redeploying and restarting the entire application.
- The system cannot scale individual components independently; for example, the enrollment module cannot be scaled separately from the notification module even though their workloads differ significantly.
- Modules are tightly coupled through shared in-process calls and shared database tables, which limits flexibility and future expansion.

Maintainability & Usability Issues

- Business logic is duplicated in multiple places, such as fee calculation, which is implemented differently across modules.
- The codebase is large and poorly documented, with unclear separation of responsibilities between teams.
- The user interface provides inconsistent feedback during registration, often failing to clearly explain why a registration was unsuccessful (e.g., capacity limits, missing prerequisites, or academic holds).
- There is no self-service password recovery system, which leads to a high number of support requests during peak enrollment periods.

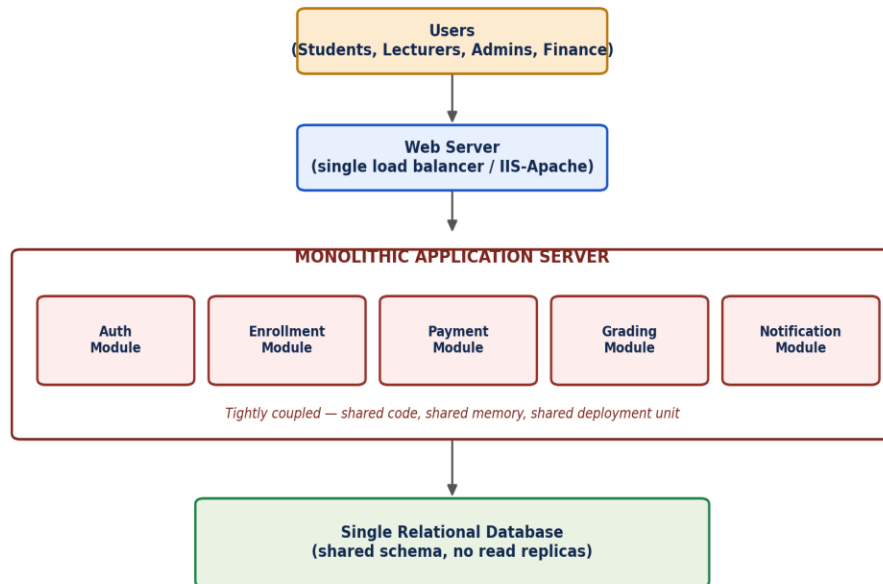
1.3 Deliverable 2: Risk Assessment Matrix

The risks in the current system are evaluated using a simple scoring method where Likelihood and Impact are rated on a scale of 1 to 5. The overall severity is calculated by multiplying these two values (Severity = Likelihood × Impact).

Risk	Category	Likelihood	Impact	Severity	Mitigation Priority
Database deadlocks during peak enrollment	Performance	5	5	25	Critical
Unauthorized grade modification (weak RBAC)	Security	3	5	15	High
Full system outage from single app-server failure	Reliability	3	5	15	High
Payment data exposure (no encryption at rest)	Security	2	5	10	High
Duplicate/incorrect fee calculation	Maintainability	4	3	12	Medium-High
Slow catalog browsing under load	Performance	4	2	8	Medium
Inconsistent enrollment status shown to student	Usability	3	3	9	Medium
Difficulty onboarding new developers (tight coupling)	Maintainability	4	2	8	Medium
Failed deployment causes full downtime	Scalability	3	4	12	Medium-High
Lack of audit trail for compliance	Security	2	4	8	Medium

1.4 Deliverable 3: Current System Architecture Diagram

Current System: Monolithic Course Registration Architecture



Single points of failure: one app tier, one DB, no isolation between modules or tenants.

Figure 1. Current Monolithic Architecture. A single application layer contains all business modules such as Authentication, Enrollment, Payment, Grading, and Notification, and all of them use one shared database. There is no separation or isolation between the modules.

1.5 Deliverable 4: UML Diagrams

Use Case Diagram

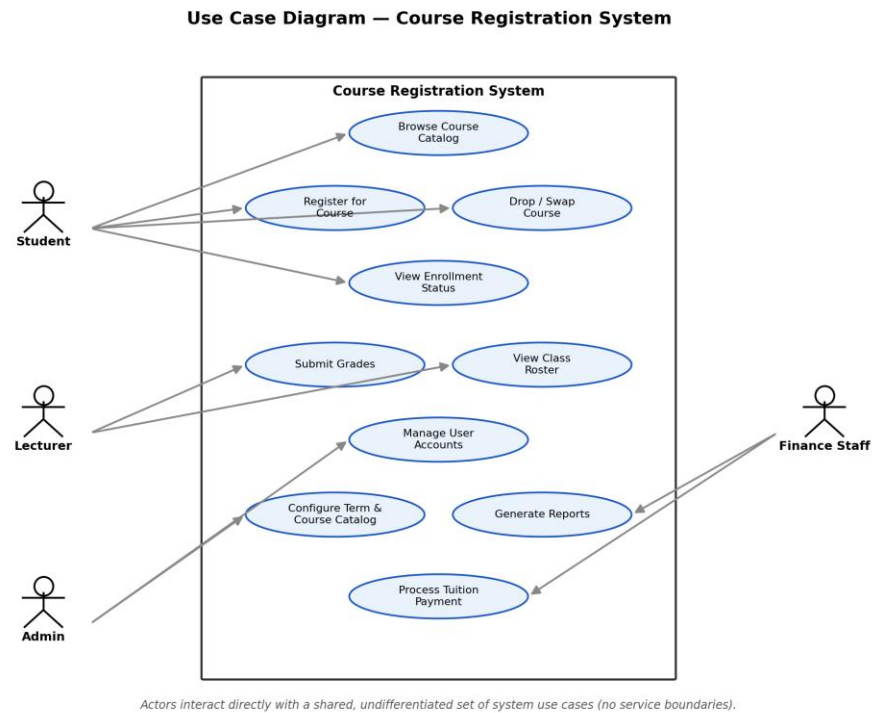


Figure 2. Use Case Diagram. The four actor types (Student, Lecturer, Admin, Finance Staff) and their primary interactions with the Course Registration System.

Activity Diagram - Course Registration Workflow

Activity Diagram — Course Registration Workflow

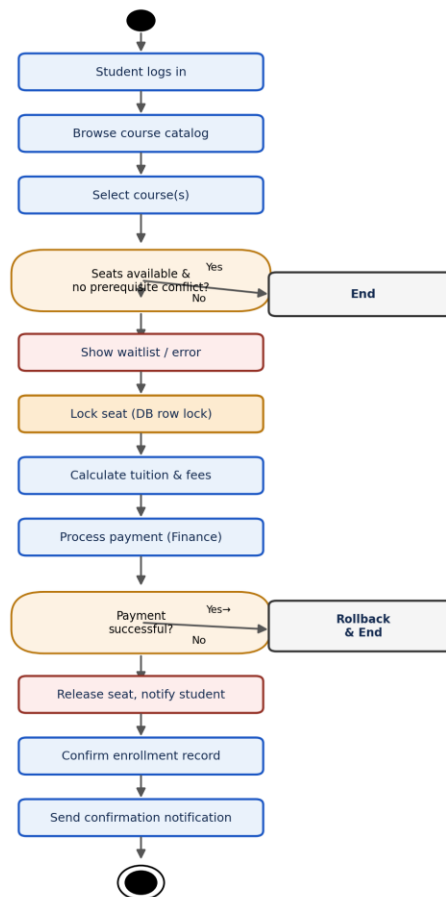


Figure 3. Activity Diagram, Course Registration Workflow. The end-to-end registration and payment workflow, including decision points for seat availability and payment success.

1.6 Critical Bottlenecks Identified

- The shared database experiences connection pool exhaustion during the first hour of course registration because many students access the system at the same time.
- Multiple students trying to register for the same course at the same time can cause row-level locking on the course and seat tables, which may lead to deadlocks.
- The enrollment process is delayed because it waits for the payment gateway to respond before completing the request. This increases response time, especially during peak traffic.
- The system runs on a single application server without a failover mechanism. If the server crashes or becomes unavailable, the entire application stops working until the issue is resolved.

Task 2 – Optimization Design and Improvement

2.1 Optimization Techniques for the Current Monolith

Before migrating to a new architecture, several improvements can be made to the existing monolithic system to reduce performance issues during the enrollment period with minimal risk.

- A read-through cache, such as Redis, can be added to store frequently accessed data like the course catalog and program requirements. Since this data does not change often, caching can reduce the load on the database.
- Common business logic, such as fee calculation, should be moved into a shared library or function. This helps avoid code duplication and ensures that the same rules are used throughout the system.
- Payment processing can be changed from a synchronous process to an asynchronous job queue with a webhook callback. This allows the enrollment request to continue without waiting for the payment gateway, improving response time.
- Database connection pooling limits and query timeouts should be configured to prevent slow queries from using all available database connections and affecting other modules.
- Read replicas can be added to handle reporting and grade-viewing requests. This reduces the workload on the primary database, allowing it to focus on enrollment and other write operations.

API Response Handling and Processing Optimization

The current API returns large amounts of data, including complete course, instructor, and department information, even when only a few fields are needed. It also performs repeated calculations, such as recalculating tuition fees every time a page is refreshed instead of only during checkout.

To improve performance, the API should return only the data required by each page using lightweight data transfer objects (DTOs). HTTP caching techniques, such as ETag and Cache-Control, can also be used so that unchanged course catalog data is loaded from the browser cache instead of requesting it from the server every time. Large results, such as course searches and class rosters, should be divided into smaller pages using pagination instead of loading all records at once. In addition, time-consuming tasks, such as generating PDF receipts, should be processed in the background instead of during the user's request.

These improvements reduce server processing time, decrease the amount of data transferred over the network, and improve the system's response time, especially during busy enrollment periods.

2.2 Deliverable 5: Optimized Database Design

The optimized database design follows the Third Normal Form (3NF) to remove data redundancy and prevent update anomalies identified in Task 1. It separates transactional data from lookup data to improve data organization and maintainability. In addition, appropriate indexes are added based on the most common system queries to improve database performance and speed up data retrieval.

Key Normalization Changes

The following changes were made to improve the database design and reduce data redundancy:

- The original **Students** table was divided into separate **Students**, **Advisors**, and **StudentAddresses** tables. This removes repeated information and makes the data easier to manage.
- A new **CourseSections** table was created to store term-specific information such as class capacity, schedule, and instructor. This prevents the same course details from being repeated every semester.
- A separate **Enrollments** table was introduced to record student enrollments using fields such as **student_id**, **section_id**, **status**, and **enrolled_at**. This replaces the previous method of storing multiple enrollments as a comma-separated list in the student record.
- Fee information was moved to a dedicated **FeeSchedules** table linked to each academic term. This removes hard-coded fee calculations from the application and makes fee updates easier to manage.

Recommended Indexes

Table	Index	Rationale
Enrollments	(student_id, term_id)	Improves the speed of retrieving a student's current enrollments, which is one of the most common queries.
Enrollments	(section_id, status)	Helps the system quickly check seat availability during course registration.
CourseSections	(term_id, course_id)	Makes course catalog searches faster when filtering by academic term.
Payments	(student_id, term_id)	Improves the performance of student billing and financial reconciliation queries.
AuditLog	(entity_type, entity_id, created_at)	Speeds up audit log searches when reviewing grade changes or payment disputes.

2.3 Deliverable 6: Improved SQL Queries

Example 1 — Original seat-availability check (full scan, no locking strategy):

```
SELECT COUNT(*) FROM enrollments
WHERE section_id = 4521 AND status = 'ENROLLED';
```

- Application then compares COUNT to capacity in a separate round trip,
- leaving a race condition between check and insert.

Optimized — atomic capacity check using a conditional update, avoiding a race condition and a second round trip:

```
UPDATE course_sections
SET seats_taken = seats_taken + 1
WHERE section_id = 4521
  AND seats_taken < capacity
RETURNING seats_taken;
```

- If zero rows are returned, the section is full; no separate SELECT needed
- and no lock is held longer than the single statement.

Example 2 — Original catalog query (no index support, filters after full join):

```
SELECT * FROM courses c, course_sections s, instructors i
WHERE c.course_id = s.course_id AND s.instructor_id = i.instructor_id
AND s.term = 'Fall12026';
```

Optimized — explicit joins, selected columns, and index-aligned filter:

```
SELECT c.course_code, c.title, s.section_number,
       s.capacity, s.seats_taken, i.full_name AS instructor
FROM course_sections s
JOIN courses c ON c.course_id = s.course_id
JOIN instructors i ON i.instructor_id = s.instructor_id
WHERE s.term_id = :term_id
ORDER BY c.course_code;
-- Backed by index on course_sections(term_id, course_id)
```

2.4 Deliverable 7: Optimization Report

Summary of proposed optimizations and their expected effect:

Optimization	Target Bottleneck	Expected Effect
Redis cache for catalog reads	Repeated requests for the same course catalog data	Reduces the number of database read operations and can lower database load by approximately 60–80% during periods of high browsing activity.
Atomic UPDATE...RETURNING for seat allocation	Deadlocks and race conditions during course registration	Prevents conflicts when multiple students register for the same course at the same time and reduces database lock duration.
Async payment processing	Request threads waiting for payment confirmation	Allows the application to process more user requests by handling payments in the background, improving system throughput during peak usage.
Read replicas for reporting	Competition between transactional and reporting queries	Separates reporting workloads from enrollment transactions, improving overall system performance.
Composite indexes	Slow queries caused by full table scans	Speeds up frequently used queries by allowing the database to find records more efficiently through indexes.

2.5 Deliverable 8: Refactored Pseudocode

The original enrollment process contained the fee calculation directly within the enrollment function, resulting in duplicated business logic and making future updates more difficult.

```
function enrollStudent(studentId, sectionId):
    if hasHold(studentId): return error
    fee = baseTuition * creditHours * 1.05 // rounding rule A
    charge(studentId, fee)
    insertEnrollment(studentId, sectionId)
```

The refactored version separates responsibilities by moving the fee calculation into a shared **FeeCalculator** component and handling payment asynchronously. It also includes proper error handling

for account holds, full sections, and payment failures, making the code easier to maintain and more reliable.

```
function enrollStudent(studentId, sectionId):
    if hasHold(studentId):
        return Result.fail('ACCOUNT_HOLD')

    reserved = trySeatReservation(sectionId)    // atomic UPDATE...RETURNING
    if not reserved:
        return Result.fail('SECTION_FULL')

    fee = FeeCalculator.calculate(studentId, sectionId)    // single source of truth
    paymentResult = PaymentService.chargeAsync(studentId, fee)

    if not paymentResult.accepted:
        releaseSeatReservation(sectionId)
        return Result.fail('PAYMENT_REJECTED')

    recordEnrollment(studentId, sectionId, status='PENDING_CONFIRMATION')
    return Result.ok('ENROLLMENT_PENDING')
```

Task 3 – Software Reengineering and Architecture Redesign

3.1 Bounded Contexts

Using the Domain-Driven Design (DDD) approach, the existing monolithic system was divided into five bounded contexts. Each bounded context represents a specific business function and can be developed and deployed as an independent microservice. This separation improves scalability, maintainability, and system reliability.

Bounded Context	Responsibility	Owns Data
Authentication	Manages user authentication, login, multi-factor authentication (MFA), token generation, and role-based access control (RBAC).	Users, Roles, Credentials, Sessions
Enrollment	Handles the course catalog, course sections, seat allocation, and student enrollment.	Courses, CourseSections, Enrollments
Payment	Manages fee calculation, tuition payments, refunds, and financial reconciliation.	Invoices, Payments, FeeSchedules
Notification	Sends email, SMS, and push notifications for confirmations, reminders, and alerts.	NotificationTemplates, DeliveryLog
Academic Record	Maintains student grades, transcripts, GPA, and degree progress.	Grades, Transcripts, DegreeRequirements

3.2 Deliverable 9: Microservice Architecture Diagram

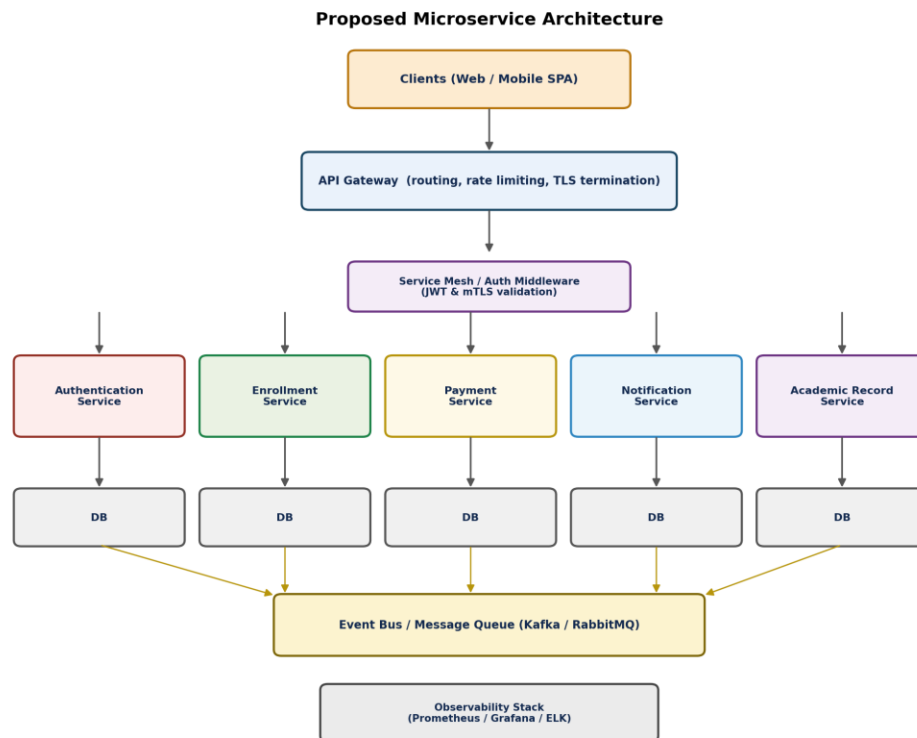


Figure 4. Proposed Microservice Architecture. Clients reach services through an API Gateway and Service Mesh; the five services (Authentication, Enrollment, Payment, Notification, Academic Record) each own their own database and communicate asynchronously via an Event Bus.

3.3 Deliverable 10: Service Decomposition Report

The monolithic application is divided into separate microservices, with each service responsible for a specific business function. This separation improves scalability, maintainability, and reduces dependencies between different parts of the system.

Authentication Service

The Authentication Service is responsible for managing user identity, verifying login credentials, and supporting multi-factor authentication (MFA). After successful authentication, it generates and issues signed JWT tokens. Other services verify these tokens using a shared public key instead of contacting the Authentication Service for every request. This reduces unnecessary communication and prevents the Authentication Service from becoming a performance bottleneck.

Enrollment Service

The Enrollment Service manages the course catalog, seat allocation, and student enrollment process. When a student starts the enrollment process, the service publishes a `SeatReserved` event to the event

bus. It then listens for PaymentConfirmed or PaymentFailed events to either complete the enrollment or release the reserved seat if the payment is unsuccessful.

Payment Service

The Payment Service is responsible for calculating tuition fees and processing payments through an external payment gateway. After processing a payment, it publishes either a PaymentConfirmed or PaymentFailed event. The Enrollment Service does not call the Payment Service directly, which reduces service dependency and prevents payment delays from affecting the enrollment process.

Notification Service

The Notification Service listens to events such as EnrollmentConfirmed, PaymentFailed, and GradePosted from the event bus. Based on these events, it sends notifications through email, SMS, or push messages. Since it works through events, it is fully separated from the main request flow of other services, which improves system performance and reduces dependencies.

Academic Record Service

The Academic Record Service manages student grades and transcripts. It listens to EnrollmentConfirmed events to update which courses a student is officially enrolled in. It also provides read-only APIs that are used to generate transcripts and check degree progress. This keeps academic data organized and separate from other services.

3.4 Deliverable 11: API Design Specification

The following table shows the main REST API endpoints designed for the microservices. A full OpenAPI specification is maintained separately for each service.

Method & Path	Service	Description
POST /auth/login	Authentication	Authenticates user credentials, triggers MFA if needed, and returns a short-lived JWT token.
POST /auth/refresh	Authentication	Generates a new access token using a refresh token.
GET /catalog/sections?term=	Enrollment	Returns a list of available course sections for a given term. This endpoint is optimized for read-heavy and cache-friendly access.
POST /enrollments	Enrollment	Creates an enrollment request and reserves a seat with a PENDING status until payment is completed.
DELETE /enrollments/{id}	Enrollment	Allows a student to drop a course and release the reserved seat.
POST /payments/charge	Payment	Charges the student's saved payment method for a pending enrollment.
GET /records/{studentId}/transcript	Academic Record	Retrieves the official student transcript.
POST /notifications/send	Notification	Internal endpoint used only by backend services to send notifications; not accessible directly by users.

Design Conventions

All external APIs are versioned using `/v1/` in the URL structure. Except for the login endpoint, all requests require a valid JWT token for authentication. Standard HTTP status codes are used to represent success and error responses, and error messages follow the **problem+json** format.

Communication between services uses mutual TLS (mTLS) for security. Instead of directly calling other services for updates, changes are shared through an event bus. This helps prevent cascading failures and reduces tight coupling between services.

3.5 Deliverable 12: DevSecOps Pipeline Proposal

Each microservice is developed, tested, and deployed independently using a shared CI/CD pipeline template. This approach ensures consistency across services while still allowing independent releases.

- **Source Control:** The system uses trunk-based development with short-lived feature branches. Every change must go through a pull request and code review before being merged.
- **CI Stage:** On every commit, automated unit tests are executed along with static code analysis (SAST), dependency vulnerability checks (SCA), and code coverage validation.
- **Build Stage:** Each service is packaged into an immutable container image. The image is tagged using the commit SHA and stored in a private container registry.
- **Security Stage:** Container images are scanned for known vulnerabilities (CVEs) before deployment. Secrets are not stored in images; instead, they are securely injected at runtime using a vault system.
- **CD Stage:** Deployments are first released to a staging environment, where integration and contract tests are executed. Production deployment is done using canary or blue-green strategies to reduce risk.
- **Runtime Environment:** Services run in Kubernetes, with auto-scaling configured per service based on metrics such as CPU usage and queue depth.
- **Monitoring:** The system uses centralized logging (ELK/OpenSearch), metrics monitoring (Prometheus/Grafana), and distributed tracing (OpenTelemetry). Automatic rollback is triggered if error rates increase beyond a defined threshold.

Task 4 – Security and Reliability Engineering

4.1 Deliverable 13: Security Assessment Report

The legacy system contains several security weaknesses. These issues are linked to their root causes and categorized based on severity.

Vulnerability	Root Cause	Severity
Weak password storage	Passwords are not stored using secure hashing methods such as bcrypt or Argon2	Critical
No multi-factor authentication	The system only uses single-factor login authentication	High
Inconsistent authorization checks	Role-based access control is handled separately in different modules instead of a central system	Critical
Sensitive data unencrypted at rest	Payment and personal identity data are stored in plain text	Critical
Inconsistent TLS enforcement	Some internal service communications are not encrypted	High
Insufficient audit logging	There is no secure or tamper-proof logging of important actions like grade or payment updates	Medium
No rate limiting on login	The login system is exposed to brute-force and credential-stuffing attacks	Medium

4.2 Deliverable 14: Threat Model Diagram

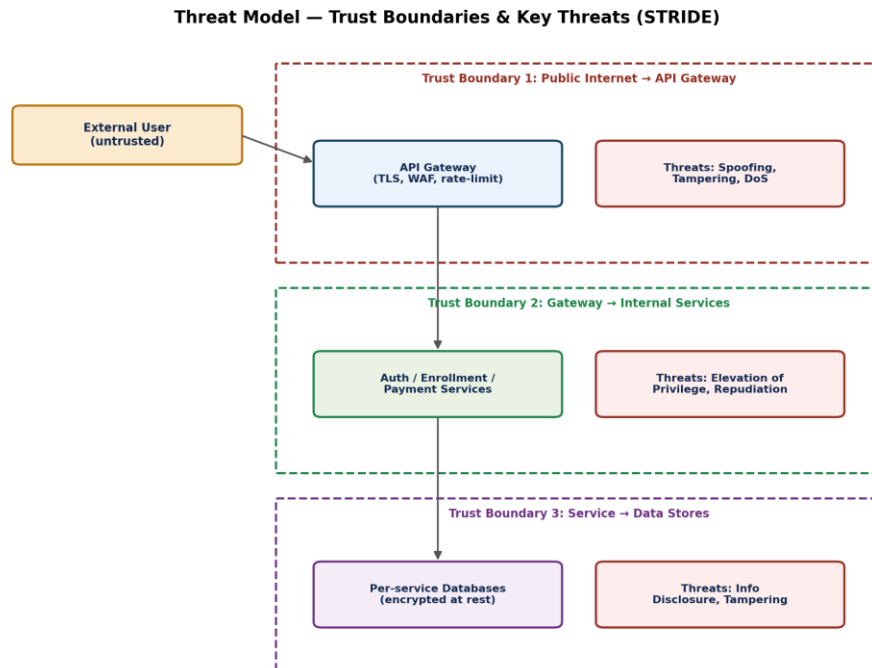


Figure 5. Threat Model - Trust Boundaries (STRIDE).

This figure shows the main security trust boundaries in the system, based on the STRIDE model. It highlights possible threats between the public internet, the API gateway and backend services, and the communication between services and the database layer.

4.3 Deliverable 15: Security Improvement Plan

The following improvements are proposed to strengthen the system's security, especially in authentication, authorization, and data protection.

Authentication & Authorization

- Role-Based Access Control (RBAC) should be enforced centrally within the Authentication Service. Each service should only validate signed JWT tokens that include user roles, instead of implementing separate authorization logic.
- Multi-Factor Authentication (MFA) should be required for all staff users such as lecturers, administrators, and finance staff. For students, MFA can be optional. It can be implemented using TOTP codes or push notification approval.
- The principle of least privilege should be applied. For example, finance staff should only be able to manage payments and should not access or modify grades, while lecturers should only be allowed to update grades for the courses they teach.

Encryption

- Sensitive data such as payment information and national ID numbers should be encrypted at rest using AES-256 encryption. Encryption keys should be managed through a dedicated Key Management Service (KMS) instead of being stored in the application code.

- All external communication should use TLS 1.2 or higher, and internal service-to-service communication should use mutual TLS (mTLS), preferably managed through a service mesh.
- Encryption keys and service credentials should be rotated regularly and immediately changed if there is any suspicion of a security breach.

Audit Logging & Monitoring

- The system should generate an immutable, append-only audit log for all important actions such as grade updates, payment transactions, and permission changes. Each log entry should include the user (actor), timestamp, and both the previous and updated state.
- All logs should be collected in a centralized logging system that supports SIEM (Security Information and Event Management). Alert rules should be configured to detect unusual behavior, such as repeated login failures or large numbers of grade changes from the same user or IP address.
- Audit logs should be stored according to the university's retention and compliance policies. Access to these logs should be strictly controlled, and any access to the logs should also be recorded.

4.4 Fault Tolerance and Recovery

- The single application server should be replaced with multiple stateless service instances behind an API Gateway or load balancer. This ensures that the failure of one instance does not bring down the entire system.
- A circuit breaker pattern should be used for communication between services (for example, Enrollment and Payment). This prevents failures in one service from affecting others and allows the system to degrade gracefully under load.
- Event processing should be made idempotent because the event bus may deliver messages more than once. This ensures that retries do not result in duplicate actions such as double payments or duplicate enrollments.
- Automated database backups should be maintained with point-in-time recovery support. Backup restoration should be tested regularly to ensure that recovery works correctly when needed.

4.5 Deliverable 16: Incident Response Checklist

- **Detect:** The system detects incidents through automated alerts such as error rate spikes, unusual login patterns, or data exfiltration attempts. Incidents can also be reported manually by users or staff.
- **Triage:** The on-call engineer evaluates the issue, assigns a severity level (P1–P4), and identifies whether it is related to security, system availability, or data integrity.
- **Contain:** The affected service is isolated to prevent further impact. This may include revoking compromised credentials or isolating affected containers, while preserving forensic evidence for investigation.
- **Eradicate:** The root cause is removed by applying fixes, rotating secrets, deploying patches, and ensuring that unauthorized access is fully eliminated.
- **Recover:** The system is restored to normal operation, data integrity is verified, and the service is closely monitored to ensure the issue does not happen again.

- **Notify:** Relevant stakeholders, including students, staff, and compliance teams, are informed according to the university's data breach notification policy and legal requirements.
- **Review:** A blameless post-incident review is conducted within five business days. The results are used to update system documentation, monitoring rules, and testing procedures to prevent similar incidents in the future.

Task 5 – Testing, Validation, and Evaluation

5.1 Validation and Testing Strategy

- The testing strategy follows the verification phase of the system development lifecycle. The main goal is to ensure that the reengineered system is functionally correct, performs well under high load, and meets security requirements before it is deployed.
- Testing is carried out in multiple layers, including unit testing, integration testing, regression testing, performance testing, and security testing. All these tests are automated and integrated into the CI/CD pipeline described in Task 3, so they run continuously during development and before each release.

5.2 Deliverable 17: Test Checklist

The table below shows the main test cases used to validate the system across different categories, along with the expected results.

Test Type	Example Test Case	Pass Criterion
Functional	Student enrolls in a section with available seats	Enrollment is created successfully and the seat count decreases by 1
Functional	Student tries to enroll in a full section	The request is rejected with a SECTION_FULL error and no over-allocation occurs
Integration	Enrollment → Payment → Notification flow	A PaymentConfirmed event finalizes the enrollment and triggers a notification
Regression	Fee calculation for different scholarship/discount levels	The calculated fee matches expected values after every system update
Performance	1,000 students attempt enrollment in the same section at once	95th percentile response time is under 500ms, with no deadlocks or overbooking
Performance	5,000 concurrent users browsing the course catalog	Cache hit rate stays above 90% and database load remains stable
Security	Student role tries to modify grades	Request is denied with HTTP 403 and the attempt is logged in the audit system
Security	Simulated brute-force login attack	Rate limiting or account lockout is triggered after a defined threshold
Reliability	One Enrollment service instance is terminated during load	Requests are automatically routed to healthy instances with no failures

5.3 Deliverable 18: Validation Report

This validation plan explains how the proposed reengineered system will be evaluated to ensure it meets the objectives of the project. The main goals are to improve system performance during the enrollment period, eliminate database inconsistencies, strengthen security, and provide a better user experience.

Since the system has not been implemented or tested as part of this project, no actual test results are available. Therefore, the **Expected Outcome** column describes the criteria that will be used to determine whether each test passes or fails once the system is developed and tested.

The table below describes the project objectives, the validation methods used to evaluate them, and the expected outcomes that will determine whether the system meets the required standards.

Objective	Validation Method	Expected Outcome (Pass Criterion)
Eliminate performance issues during the enrollment period	Perform a load test using three times the historical peak number of concurrent users	The system maintains response times within the agreed Service Level Agreement (SLA) throughout the test.
Remove database inconsistencies	Conduct a database schema audit using a 3NF checklist and anomaly test cases	No update, insert, or delete anomalies are found in the database schema.
Close the security gaps identified in Task 4	Perform penetration testing on RBAC, MFA, and encryption controls	No critical security vulnerabilities are found, and any medium-level issues are documented for remediation.
Improve the usability of registration error messages	Conduct usability testing with a group of student users	Students rate the error messages as clearer and easier to understand than those in the legacy system.
Ensure independent scalability of services	Increase the number of Enrollment service instances during a load test	Scaling the Enrollment service does not noticeably affect the performance of the Notification or Academic Record services.

5.4 Deliverable 19: Performance Comparison Metrics

Important Note on Data

This project is a design and planning exercise, so the proposed reengineered system has not been implemented or tested. In addition, no real performance data or system logs from the existing legacy system were available, as explained in the **Assumptions** section of the introduction.

For this reason, the values presented in the following sections are estimated rather than actual measurements. The legacy system figures are based on typical performance issues found in monolithic systems with the bottlenecks identified in Task 1. The projected values for the reengineered system represent the expected improvements based on commonly used software engineering practices, such as

horizontal scaling, atomic seat reservation, asynchronous payment processing, and independent microservices.

These estimated values should be replaced with real performance measurements after the proposed system has been implemented and evaluated through the testing strategy described in Sections 5.2 and 5.3.

Estimated legacy baseline vs. projected reengineered-architecture targets:

The table below compares the estimated performance of the legacy system with the expected performance of the proposed reengineered system. Since the new architecture has not yet been implemented, these values represent projected targets rather than actual test results.

KPI	Legacy System (estimated)	Reengineered System (projected target)	Projected Improvement
Peak enrollment 95th-percentile response time	Approximately 4–5 seconds	Approximately 0.3–0.5 seconds	Around 90% reduction in response time
Max concurrent enrollment requests before errors	Approximately 500–800 requests	More than 10,000 requests through horizontal scaling	> More than 10 times improvement (target)
Database deadlocks per peak enrollment day	Approximately 40–60 per day	Close to zero using atomic seat reservation	More than 95% reduction
Mean time to recover from a service failure	Approximately 30–60 minutes with manual restart	Less than 2 minutes through automatic restart and traffic rerouting	Around 95% reduction
Deployment frequency	Monthly deployment of the entire system	Multiple deployments per day for individual services	Significant improvement in deployment flexibility
Critical/high security findings	5 issues identified in Task 4	Target of zero after remediation	Expected 100% remediation, subject to penetration testing

These projected targets should be verified through actual testing before they are considered achieved. This includes performance benchmarking with load tests, fault tolerance testing using chaos or failure injection, and an independent penetration test to confirm the system's security. The final results should only be reported after the system has been implemented and these tests have been successfully completed.

5.5 Final Recommendations and Future Improvement Opportunities

Based on the proposed reengineering design, the following recommendations can help ensure a successful migration and support future system improvements:

- The migration should be carried out in phases by gradually replacing the monolithic system with microservices. Starting with lower-risk services such as **Authentication** and **Notification** will reduce migration risks and minimize disruption.

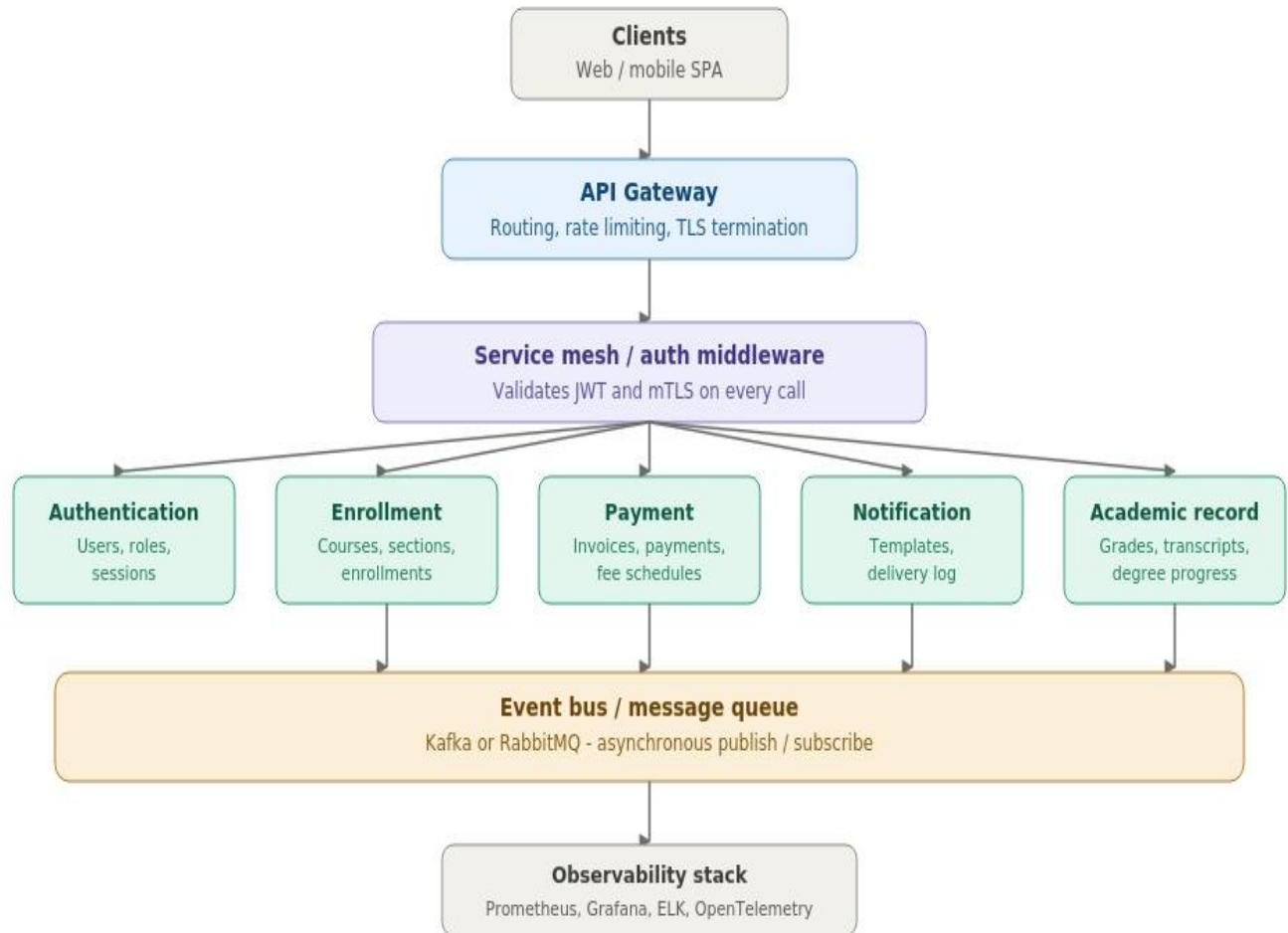
- Automated contract testing should be introduced early in the development process to ensure reliable communication between microservices, especially when using event-driven architecture.
- A dedicated Platform or DevSecOps team should manage shared infrastructure such as the CI/CD pipeline, service mesh, monitoring tools, and logging systems. This allows development teams to focus on building their services without duplicating infrastructure work.
- If client applications need data from multiple services in a single request, a **GraphQL** or **Backend-for-Frontend** layer can be added in front of the API Gateway to improve efficiency and simplify client development.
- Capacity planning should be reviewed before each academic term because student enrollment demand changes throughout the year. Auto-scaling settings should be adjusted based on historical usage and expected demand.
- The audit logging and monitoring system should be further enhanced to support compliance reporting, helping the university meet regulatory requirements while protecting sensitive academic and financial information.

References

The design recommendations presented in this report are based on established industry standards, security guidelines, and software engineering best practices. Where recommendations are based on widely accepted frameworks or architectural patterns, such as Role-Based Access Control (RBAC), the OWASP Top 10, and the microservices architecture, the relevant sources are listed below instead of being cited throughout the report.

- OWASP Foundation. *OWASP Top 10 – Web Application Security Risks*. Available at: <https://owasp.org/www-project-top-ten/>
- OWASP Foundation. *OWASP Application Security Verification Standard (ASVS)*. Available at: <https://owasp.org/www-project-application-security-verification-standard/>
- National Institute of Standards and Technology (NIST). *SP 800-63B – Digital Identity Guidelines: Authentication and Lifecycle Management*. Available at: <https://pages.nist.gov/800-63-3/>
- National Institute of Standards and Technology (NIST). *SP 800-53 – Security and Privacy Controls for Information Systems and Organizations*. Available at: <https://csrc.nist.gov/publications/detail/sp/800-53/rev-5/final>
- National Institute of Standards and Technology (NIST). *Computer Security Incident Handling Guide (SP 800-61)*. Available at: <https://csrc.nist.gov/publications/detail/sp/800-61/rev-2/final>
- Fielding, R. *Architectural Styles and the Design of Network-based Software Architectures (REST)*. Available at: <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- Newman, S. *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media.
- Richardson, C. *Microservices Patterns: With Examples in Java*. Manning Publications. Additional resources: <https://microservices.io/patterns/>
- Kubernetes Documentation. *Concepts and Best Practices*. Available at: <https://kubernetes.io/docs/concepts/>

Proposed system: overall architecture and registration workflow



Course registration workflow (event-driven)

1. Student submits an enrollment request through the API Gateway, authenticated via a JWT from the Authentication Service.
 2. Enrollment Service atomically reserves a seat and publishes a `SeatReserved` event to the Event Bus.
 3. Payment Service consumes the event, charges the student, and publishes `PaymentConfirmed` or `PaymentFailed`.
 4. Enrollment Service listens for that result: it finalizes the enrollment, or releases the seat if payment failed.
 5. Notification Service sends a confirmation, and Academic Record Service updates the student's official course list.
- Services never call each other directly - all cross-service updates flow asynchronously through the Event Bus.